// GUIDA GRATIS · CLAUDE CODE DA ZERO

CLAUDE CODE DA ZERO IN *8 passi.*

L'ordine in cui lo imparerei se ripartissi oggi. Ogni passo ha: cosa fai, perché, il comando esatto, cosa devi vedere e gli errori più comuni.

1 · **Installalo**	app desktop o terminale	pag. 02
2 · **Plan mode**	prima il piano, poi i file	pag. 03
3 · **CLAUDE.md**	le regole che legge ogni volta	pag. 04
4 · **Contesto**	/context, /clear, /compact	pag. 05
5 · **Skill**	un lavoro ripetuto in un file	pag. 06
6 · **MCP**	collega Notion e le altre app	pag. 07
7 · **Subagenti + hook**	deleghe e notifiche	pag. 08
8 · **Senza di te**	compiti con orario	pag. 09

// COSA TI SERVE

01 Un account che include Claude Code
Piano Pro, Max, Ultra, Team o Enterprise, oppure un account Claude Console con credito API. Il piano Free non è nell'elenco.

02 Un computer
macOS, Windows o Linux. Per il passo 8 nel cloud serve anche un repository GitHub.

03 Una cartella di prova
Con qualche file dentro, non un progetto importante: ci farai tutti gli esercizi.

// COME USARLA

Fai i passi in ordine, sempre nella stessa cartella. I riquadri «cosa devi vedere» sono **esempi**: il testo esatto cambia con la versione. Comandi e link presi dalle documentazioni ufficiali e controllati il 04/11/2026.

// PASSO 01 · INSTALLALO

01 INSTALLALO. *Il terminale non serve.*

Perché: è il punto di partenza di tutto il resto. La strada più semplice è l'app desktop di Claude, dove Claude Code è la tab **Code**. Se preferisci il terminale, è un comando.

01 Strada A · app desktop
Scarica l'app Claude (macOS, Windows, Linux in beta), accedi, clicca la tab **Code** e scegli la cartella di prova.

02 Strada B · terminale
macOS, Linux, WSL: il comando qui sotto. Windows PowerShell: `irm https://claude.ai/install.ps1 | iex`.

03 Primo avvio
Entra nella cartella con `cd` e scrivi `claude`: la prima volta ti fa accedere dal browser.

04 Il corso gratuito
Claude Code 101 su Anthropic Academy: 12 lezioni più un quiz, circa 1h30. Copre anche i passi 2-7 di questa guida.

// COMANDO (macOS / Linux / WSL)

```
curl -fsSL https://claude.ai/install.sh | bash
claude --version
```

Alternative: `brew install --cask claude-code` (non si aggiorna da solo) · `winget install Anthropic.ClaudeCode`

// COSA DEVI VEDERE (esempio)

```
$ claude --version
<numero di versione> (Claude Code)
$ claude
> Fammi un riassunto dei file in questa cartella.
```

Errori comuni — «command not found» subito dopo l'installazione: chiudi e riapri il terminale. Su Windows installa prima Git for Windows. Account Free: Claude Code non parte, serve uno dei piani della pagina 1.

Consiglio — se il terminale ti blocca, parti dall'app desktop: tutti gli esercizi di questa guida funzionano uguali nella tab Code.

FONTI · code.claude.com/docs/en/quickstart · code.claude.com/docs/en/desktop · academy.claude.com/courses/claude-code-101

// PASSO 02 · PLAN MODE

02 PRIMA IL PIANO, *poi i file.*

Perché: in plan mode Claude legge il progetto e ti scrive cosa vuole fare **senza modificare nulla**. Tu lo correggi, e solo dopo lo lasci costruire. È il modo più rapido per non buttare lavoro.

01 Attivalo
Premi `Shift+Tab`: i modi girano normale → accetta modifiche → plan. Il modo attivo è scritto in basso.

02 Oppure parti già in plan
Da terminale `claude --permission-mode plan`, o scrivi `/plan` davanti a una singola richiesta.

03 Correggi il piano
Rispondi come a un collega: «il punto 3 no, usa il file che c'è già». Fallo prima di approvare.

04 Approva
Quando il piano ti torna lo approvi: solo da lì Claude modifica i file.

// PROMPT DI PROVA

`/plan` Aggiungi una pagina contatti al sito in questa cartella. Prima dimmi quali file tocchi e perché.

// COSA DEVI VEDERE (esempio)

```
⏸ plan mode on
Piano:
1. Creo contatti.html con lo stesso stile di index.html
2. Aggiungo il link «Contatti» nel menu di index.html
3. Nessun altro file modificato
Procedo?
```

Errori comuni — premere `Shift+Tab` una volta di troppo e finire in un altro modo: guarda l'indicatore in basso prima di scrivere. Approvare senza leggere: il piano serve solo se lo correggi. Per un errore di battitura non serve, usalo sui lavori che toccano più file.

Consiglio — aggiungi al prompt «fammi domande se qualcosa non è chiaro»: le risposte che dai prima del piano sono quelle che ti evitano di rifare il lavoro dopo.

FONTI · code.claude.com/docs/en/permission-modes

// PASSO 03 · CLAUDE.MD

03 SCRIVILO UNA VOLTA, *non ripeterlo più.*

Perché: CLAUDE.md è un file che Claude legge ogni volta che parte. Quello che ci scrivi (come lavori, cosa non toccare) vale in ogni sessione senza doverlo ridire.

01 Fallo scrivere a lui
Nella cartella lancia `/init`: legge il progetto e crea un CLAUDE.md di partenza. Se esiste già, propone miglioramenti.

02 Dove vive
`./CLAUDE.md` vale per il progetto (si condivide col team). `~/.claude/CLAUDE.md` vale per te, in tutti i progetti.

03 Cosa metterci
Solo quello che correggi sempre: lingua, stile, comandi, cartelle vietate. Poche righe chiare battono tre pagine.

04 Uno per ruolo
Da noi ogni agente ha la sua cartella con il suo CLAUDE.md: aperto lì, Claude sa già come vogliamo le cose.

// ESEMPIO DI CLAUDE.md

```
# Regole di questo progetto
- Rispondi sempre in italiano
- Prima di modificare più file, mostrami il piano
- Non toccare la cartella /clienti
- Dopo ogni modifica lancia i test e dimmi l'esito
```

// COSA DEVI VEDERE

Apri una sessione nuova e chiedi «**Quali regole segui in questa cartella?**»: deve elencare le tue. Con `/memory` vedi quali CLAUDE.md sono caricati e li apri per modificarli.

Errori comuni — scriverlo tutto prima di aver lavorato: parti senza, e aggiungi una riga ogni volta che correggi Claude sulla stessa cosa. Metterci password o chiavi: il CLAUDE.md di progetto finisce nel repository e lo vede chiunque.

Consiglio — quando correggi Claude due volte sulla stessa cosa, digli «aggiungi questa regola al CLAUDE.md»: la scrive lui, tu rileggi solo la riga.

FONTI · code.claude.com/docs/en/memory

// PASSO 04 · IL CONTESTO

04 LA SUA MEMORIA *si riempie.*

Perché: il contesto è la memoria di lavoro della sessione: i tuoi messaggi, i file letti, le risposte. Ha un limite, e quando si avvicina Claude riassume da solo la parte vecchia. Gestirlo tu rende le risposte più precise.

01 Guarda quanto è pieno
`/context` mostra una griglia colorata: quanto occupano istruzioni, strumenti, CLAUDE.md, messaggi, e quanto spazio resta.

02 Cambi lavoro? /clear
Riparte una conversazione vuota. Il CLAUDE.md resta: lo rilegge.

03 Stesso lavoro, troppo lungo? /compact
Riassume la conversazione e continua. Puoi dirgli su cosa concentrarsi.

04 Una sessione, un compito
Tre lavori diversi nella stessa conversazione la riempiono di cose che non servono a nessuno dei tre.

// COMANDI

```
/context
/clear
/compact concentrati sul bug del login
```

// COSA DEVI VEDERE (esempio)

```
/context
■ ■ ■ ■ ■ ■ □ □ □ □ □ □ □ □ □ □   uso del contesto
istruzioni · strumenti · memoria · messaggi · spazio libero

/compact  → «Conversazione riassunta», e continui da lì
```

Errori comuni — usare `/clear` a metà di un lavoro e perdere il filo: lì serve `/compact`. Ripetere in chat regole che valgono sempre: mettile nel CLAUDE.md (passo 3), così sopravvivono anche a `/clear`.

Consiglio — prima di un `/compact` scrivi cosa deve tenere («concentrati sulle decisioni prese»): il riassunto conserva quello che conta per te.

FONTI · code.claude.com/docs/en/commands · code.claude.com/docs/en/context-window

// PASSO 05 · LE SKILL

05 UN LAVORO CHE RIPETI, *in un file.*

Perché: una skill salva un procedimento una volta sola. Claude la carica da solo quando la tua richiesta corrisponde alla descrizione, oppure la lanci tu con `/nome-skill`.

01 Com'è fatta
Una cartella con dentro `SKILL.md`: in testa, tra due righe `---`, nome e descrizione; sotto, le istruzioni.

02 Dove va
`~/.claude/skills/nome/SKILL.md` per tutti i tuoi progetti, `.claude/skills/nome/SKILL.md` solo per questo.

03 La descrizione decide
È quella che Claude legge per capire quando usarla: scrivi **quando** serve, non solo cosa fa.

04 Parti da quelle ufficiali
Il repository **anthropics/skills** (oltre 180mila stelle su GitHub) ha skill pronte, per esempio per documenti PDF, Word, Excel e PowerPoint.

// ESEMPIO · ~/.claude/skills/riassunto-riunione/SKILL.md

```
---
name: riassunto-riunione
description: Usala quando incollo gli appunti di una riunione e chiedo un riassunto.
---
Scrivi tre blocchi: decisioni prese, cose da fare con il responsabile,
domande aperte. Massimo 10 righe, in italiano.
```

// INSTALLARE QUELLE UFFICIALI (dal README del repository)

```
/plugin marketplace add anthropics/skills
/plugin install document-skills@anthropic-agent-skills
```

Poi `/skills` elenca quelle disponibili. Corso gratuito: **Introduction to agent skills**, 6 lezioni, circa 1 ora.

Errori comuni — descrizione vaga («aiuta con i documenti»): Claude non capisce quando caricarla. File con un nome diverso da `SKILL.md` o fuori dalla sua cartella: non viene trovato.

Consiglio — non scrivere la skill da zero: dopo un lavoro riuscito chiedi a Claude «trasforma quello che abbiamo appena fatto in una skill», poi rileggi la descrizione.

FONTI · code.claude.com/docs/en/skills · github.com/anthropics/skills · academy.claude.com/courses/introduction-to-agent-skills

// PASSO 06 · MCP

06 COLLEGALO *alle tue app.*

Perché: MCP è il modo standard per collegare Claude Code ad altre app, come Notion, GitHub o Gmail. Una volta collegato, Claude legge e scrive lì dentro mentre lavora, senza copia-incolla.

01 Aggiungi il server
Un comando nel terminale, fuori dalla sessione di Claude. Per Notion è quello qui sotto, uguale nella doc di Claude Code e in quella di Notion.

02 Accedi
Apri Claude e lancia `/mcp`: scegli notion e fai il login dal browser. Nessuna chiave da copiare.

03 Controlla
`claude mcp list` mostra lo stato di ogni server collegato.

04 Per chi vale
`--scope local` (predefinito: solo tu, in questo progetto), `project` (crea `.mcp.json` da condividere), `user` (tutti i tuoi progetti).

// COMANDO (documentazione ufficiale)

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

// COSA DEVI VEDERE (esempio)

```
$ claude mcp list
notion: https://mcp.notion.com/mcp (HTTP) - ✔ Connected
```

Prima del login vedrai «**! Needs authentication**»: è normale, fai `/mcp`.

Errori comuni — lanciare `claude mcp add` dentro la chat invece che nel terminale. Notion MCP funziona su tutti i piani Notion, anche il gratuito, ma alcune funzioni (come la ricerca nelle fonti collegate) sono solo Business ed Enterprise, e gli ospiti di un workspace non possono collegarsi.

Consiglio — collega un solo server alla volta e fai una domanda semplice («elenca le mie ultime 5 pagine Notion»): se risponde, il collegamento c'è.

FONTI · code.claude.com/docs/en/mcp · developers.notion.com/docs/get-started-with-mcp

// PASSO 07 · SUBAGENTI E HOOK

07 DELEGA I PEZZI, *fatti avvisare.*

Perché: un subagente prende un pezzo di lavoro con un contesto tutto suo e ti riporta solo il riassunto, così la tua sessione resta pulita. Un hook è un comando che parte da solo in un momento preciso.

01 Crea un subagente
Chiedilo a Claude: «crea un subagente revisore che controlla il codice e mi dà 5 punti». Lo salva in `.claude/agents/` (progetto) o `~/.claude/agents/` (tuo).

02 Com'è fatto
Un file .md: in testa `name` e `description` (obbligatori), `tools` e `model` facoltativi; sotto, le istruzioni.

03 Il primo hook: la notifica
L'evento `Notification` scatta quando Claude aspetta te (un permesso, una risposta). Va in `~/.claude/settings.json`.

04 Controlla
`/hooks` mostra gli hook attivi. Corso gratuito: **Introduction to subagents**, 4 lezioni, 45 minuti.

// HOOK NOTIFICA PER macOS · dalla doc, testo tradotto

```
{"hooks": {"Notification": [{"matcher": "",
  "hooks": [{"type": "command",
    "command": "osascript -e 'display notification \"Claude Code ha bisogno di te\"
with title \"Claude Code\"'"}]}]}}
```

// COSA DEVI VEDERE

Quando Claude chiede un permesso compare la notifica «**Claude Code ha bisogno di te**». Su Linux il comando è `notify-send 'Claude Code' 'Claude Code ha bisogno di te'`.

Errori comuni — su Mac la notifica non arriva: in Impostazioni di Sistema → Notifiche consenti le notifiche a **Script Editor**. Una virgola sbagliata nel JSON e l'hook non parte: se il file ha già altre impostazioni, aggiungi solo il blocco `"hooks"`.

Consiglio — usa i subagenti per le ricerche lunghe («cerca in tutta la cartella dove…»): la lettura dei file resta nel loro contesto, a te arriva solo la risposta.

FONTI · code.claude.com/docs/en/sub-agents · code.claude.com/docs/en/hooks-guide · academy.claude.com/courses/introduction-to-subagents

// PASSO 08 · LAVORA SENZA DI TE

08 UN COMPITO, *un orario.*

Perché: i lavori che fai ogni giorno (rivedere il codice, preparare un riepilogo) possono partire da soli. Nel cloud di Anthropic girano anche con il portatile chiuso; nell'app desktop solo se il computer è acceso.

01 Nel cloud: anche a portatile chiuso
Si chiamano **cloud scheduled tasks** (prima «routines», ancora in anteprima). Li crei da `claude.ai/code/routines` o con `/schedule`.

02 Serve un repository GitHub
Ogni esecuzione parte da una copia nuova del repository e non vede i file del tuo computer. Intervallo minimo: 1 ora.

03 Nell'app desktop: file locali
Le attività programmate locali vedono i tuoi file, ma partono solo con l'app aperta e il computer sveglio. Al risveglio recupera l'ultima saltata.

04 Scrivi il compito per esteso
Cosa fare, dove, cosa consegnare: «ogni mattina rivedi le pull request aperte e scrivimi un riepilogo di 5 righe».

// COMANDO (esempio della doc)

```
/schedule daily PR review at 9am
```

Poi Claude ti chiede i dettagli: repository, orario, cosa fare.

// COSA DEVI VEDERE (esempio)

```
Revisione PR · ogni giorno alle 9:00
lun ✓ fatto   mar ✓ fatto   mer ✓ fatto
prossima esecuzione: domani 9:00
```

Errori comuni — aspettarsi che il task nel cloud legga i file del tuo Mac: vede solo il repository. Chiudere il portatile con un task solo locale: il computer va in stop e il task salta. Corso gratuito con una lezione sui task programmati: **Claude Code in Action**, 9 lezioni, circa 1 ora.

Consiglio — prima di programmarlo, lancia il compito a mano una volta e controlla il risultato: se non va bene a mano, non andrà bene nemmeno ogni mattina.

FONTI · code.claude.com/docs/en/routines · code.claude.com/docs/en/desktop-scheduled-tasks · academy.claude.com/courses/claude-code-in-action

// VERIFICA FINALE

COME SAI CHE *funziona.*

Fai questo giro di controllo nella cartella di prova, in una sessione nuova. Se un punto non torna, la colonna a destra ti dice a quale pagina tornare.

`claude --version`	risponde con un numero di versione	pag. 02
`Shift+Tab`	in basso compare l'indicatore plan mode	pag. 03
`«quali regole segui?»`	elenca le righe del tuo CLAUDE.md	pag. 04
`/context → /clear`	la griglia si svuota, le regole restano	pag. 05
`/skills`	compare la skill che hai scritto	pag. 06
`claude mcp list`	notion con ✔ Connected	pag. 07
`/hooks`	c'è Notification e la notifica arriva	pag. 08
`/schedule`	il task ha una prossima esecuzione	pag. 09

// LA PROVA COMPLETA

01 Un lavoro vero, piccolo
In plan mode chiedi una modifica che tocca due file. Correggi il piano almeno una volta, poi approva.

02 Le regole reggono
Dopo `/clear`, chiedi una seconda modifica: deve rispettare il CLAUDE.md senza che tu lo ripeta.

03 Il cerchio si chiude
Fagli scrivere il riepilogo del lavoro in una pagina Notion: se la pagina compare, MCP funziona.

// SE QUALCOSA NON VA

Dentro Claude Code lancia `/doctor`: controlla installazione, impostazioni e CLAUDE.md e propone le correzioni, che applica solo dopo il tuo ok. Poi rileggi la pagina ufficiale del passo: i comandi cambiano con le versioni.

// LINK UFFICIALI

TUTTO QUELLO CHE TI *serve.*

Solo documentazioni ufficiali e corsi gratuiti di Anthropic, più la pagina MCP di Notion. Tutti aperti e controllati il 04/11/2026.

Installazione	code.claude.com/docs/en/quickstart	01
App desktop	code.claude.com/docs/en/desktop	01
Plan mode	code.claude.com/docs/en/permission-modes	02
CLAUDE.md	code.claude.com/docs/en/memory	03
Comandi	code.claude.com/docs/en/commands	04
Skill	code.claude.com/docs/en/skills	05
Skill ufficiali	github.com/anthropics/skills	05
MCP	code.claude.com/docs/en/mcp	06
Notion MCP	developers.notion.com/docs/get-started-with-mcp	06
Subagenti	code.claude.com/docs/en/sub-agents	07
Hook	code.claude.com/docs/en/hooks-guide	07
Task nel cloud	code.claude.com/docs/en/routines	08
Task sul desktop	code.claude.com/docs/en/desktop-scheduled-tasks	08

// CORSI GRATUITI · ANTHROPIC ACADEMY

Claude Code 101	academy.claude.com/courses/claude-code-101	12 lezioni · 1h30
Agent skills	academy.claude.com/courses/introduction-to-agent-skills	6 lezioni · 1h
Subagents	academy.claude.com/courses/introduction-to-subagents	4 lezioni · 45 min
Claude Code in Action	academy.claude.com/courses/claude-code-in-action	9 lezioni · 1h

// RICORDA

Claude Code cambia spesso: se un comando di questa guida non va, vale la pagina ufficiale del link. Fai pratica in una cartella di prova, e collega app e account solo quando sai cosa Claude può fare lì dentro: **quello che gli dai, lo può modificare**.